

Image Scaling + Rotation

Here in this experiment , we will first use opencv to read the image in grayscale format. Then we will perform image scaling using bilinear interpolation and nearest neighbour and rotation bilinear interpolation and nearest neighbour.

Geometric Transformations

Geometric transformations modify the **spatial arrangement of pixels** in an image without changing the original pixel intensity values. They are widely used in image processing for tasks such as image alignment, augmentation, document correction, and object manipulation.

Transformation Process

A geometric transformation consists of two steps:

1. **Spatial Transformation** – Computes the new location of each pixel.
2. **Interpolation** – Estimates pixel values for non-integer coordinates created during the transformation.

1. Translation

Moves an image horizontally and/or vertically.

$$x = u + t_x, y = v + t_y \quad x = u + t_x, \quad y = v + t_y$$

where:

- t_x : horizontal shift
- t_y : vertical shift

Transformation matrix:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

2. Scaling

Changes the size of the image.

$$x = S_x u, y = S_y v \quad \text{or} \quad x = S_x u, y = S_y v$$

- $S_x, S_y > 1$: Magnification (Zoom In)
- $S_x, S_y < 1$: Minification (Shrink)

Transformation matrix:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

3. Shearing

Skews the image along one direction.

For x-direction shear:

$$x = u + f v, y = v \quad \text{or} \quad x = u + f v, y = v$$

where f is the shear factor.

Transformation matrix:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & f \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

4. Rotation

Rotates the image by an angle θ (usually about the origin).

$$\begin{aligned} x &= u \cos \theta - v \sin \theta \\ y &= u \sin \theta + v \cos \theta \end{aligned}$$

Transformation matrix:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Since rotated coordinates are generally non-integer, interpolation is required to obtain smooth images.

Interpolation

Interpolation assigns intensity values to pixels whose computed coordinates are non-integer after a geometric transformation.

Nearest Neighbor Interpolation

- Assigns the value of the closest pixel.
- Fastest method.
- Produces blocky or jagged edges.

Bilinear Interpolation

- Uses the four nearest neighboring pixels.
- Computes a weighted average based on distance.
- Produces smoother images than nearest neighbor.

Bicubic Interpolation

- Uses the sixteen nearest neighboring pixels.
- Produces the smoothest and highest-quality results.
- More computationally expensive.